

Les fondations du programmation Python

Abdallah Houmadi SOIRFANE

11 septembre 2026

1 Les fichiers : quand Python rencontre le monde réel

1.1 Le problème du jour : Youssouf perd tout

Youssouf tient son stand de beignets à Mutsamoudou. Chaque soir, il note ses ventes dans un cahier : « Lundi : 15 beignets nature, 8 chocolat. Mardi : 20 nature, 12 chocolat... »

Le problème, c'est qu'un jour, le cahier est tombé dans la mer. Et là, catastrophe : toutes les données ont disparu. Impossible de savoir s'il a fait une bonne semaine ou non.

Hamadi lui dit : « Youssouf, un ordinateur, c'est pareil qu'un cahier, mais en mieux. On peut **écrire** dans un fichier, et on peut le **relire** plus tard, même après avoir éteint la machine. »

Un **fichier**, c'est comme un cahier numérique. On peut y écrire, y lire, et il se souvient de tout, même quand on dort.

1.2 Ouvrir un fichier : la porte du cahier

En Python, pour ouvrir un fichier, on utilise la fonction `open()`. Elle prend deux arguments principaux :

- Le **nom du fichier** (avec son chemin).
- Le **mode** d'ouverture :
 - `"r"` : lire (*read*). Le fichier doit exister.
 - `"w"` : écrire (*write*). **Écrase** le contenu s'il existe déjà.
 - `"a"` : ajouter (*append*). Ajoute à la fin sans effacer.

```
1  # Ouvrir un fichier en mode ecriture (w)
2  fichier = open("ventes.txt", "w")
3  fichier.write("Lundi : 15 beignets nature\n")
4  fichier.write("Mardi : 20 beignets chocolat\n")
5  fichier.close()  # On referme le cahier
6
```

Astuce du marchand

Le mode "w" est un destructeur! Si le fichier `ventes.txt` existe déjà et qu'on l'ouvre en mode `"w"`, tout son contenu est effacé avant qu'on écrive. C'est comme arracher les pages d'un cahier avant d'écrire. Si vous voulez juste ajouter à la fin, utilisez `"a"`.

1.3 Lire un fichier : retrouver ses souvenirs

Pour lire un fichier, on l'ouvre en mode "r" et on utilise `.read()` (tout lire) ou `.readlines()` (lire ligne par ligne dans une liste).

```
1 # Lire tout le contenu
2 fichier = open("ventes.txt", "r")
3 contenu = fichier.read()
4 print(contenu)
5 fichier.close()
6
7 # Lire ligne par ligne
8 fichier = open("ventes.txt", "r")
9 lignes = fichier.readlines()
10 for ligne in lignes:
11     print(ligne.strip()) # strip() enleve le \n a la fin
12 fichier.close()
13
```

1.4 La méthode magique : with

Vous avez remarqué qu'à chaque fois, on doit faire `fichier.close()` à la fin ? C'est fatigant, et si on oublie, ça peut poser problème (le fichier reste ouvert, comme une porte qu'on a oublié de fermer).

Python a une solution élégante : le mot-clé `with`. Il ouvre le fichier, fait ce qu'on a à faire, et le referme automatiquement à la fin du bloc. Même si une erreur se produit.

```
1 # La methode propre avec "with"
2 with open("ventes.txt", "w") as fichier:
3     fichier.write("Lundi : 15 beignets nature\n")
4     fichier.write("Mardi : 20 beignets chocolat\n")
5     # Le fichier est automatiquement ferme ici
6
7 # Lecture avec with
8 with open("ventes.txt", "r") as fichier:
9     contenu = fichier.read()
10    print(contenu)
11
```

Astuce du marchand

Prenez l'habitude du "with". C'est plus court, plus propre, et plus sûr. Vous n'oublierez jamais de fermer le fichier. C'est comme avoir un petit robot qui ferme la porte derrière vous.

1.5 Le mode "a" : ajouter sans effacer

Imaginez que Youssouf veuille ajouter les ventes du mercredi sans effacer celles du lundi et du mardi. Il utilise le mode "a".

```
1 # Ajouter a la fin du fichier (append)
2 with open("ventes.txt", "a") as fichier:
3     fichier.write("Mercredi : 18 beignets nature\n")
4     fichier.write("Jeudi : 25 beignets chocolat\n")
5
6 # Verifions le contenu
7 with open("ventes.txt", "r") as fichier:
8     print(fichier.read())
9
```

1.6 Les fichiers CSV : le trésor des données

Un **fichier CSV** (Comma-Separated Values) est un fichier texte où les valeurs sont séparées par des virgules. C'est le format le plus utilisé pour stocker des données tabulaires (comme un tableau Excel).

Exemple de fichier `ventes.csv` :

```
1 jour,nature,chocolat,confiture
2 Lundi,15,8,3
3 Mardi,20,12,5
4 Mercredi,18,10,4
5
```

Chaque ligne = une journée. Chaque valeur est séparée par une virgule.

Python a un module génial pour ça : `csv`.

```
1 import csv
2
3 # Ecrire dans un fichier CSV
4 with open("ventes.csv", "w", newline="") as fichier:
5     writer = csv.writer(fichier)
6     writer.writerow(["jour", "nature", "chocolat", "confiture"])
7     writer.writerow(["Lundi", 15, 8, 3])
8     writer.writerow(["Mardi", 20, 12, 5])
9     writer.writerow(["Mercredi", 18, 10, 4])
10
11 # Lire un fichier CSV
12 with open("ventes.csv", "r") as fichier:
13     reader = csv.reader(fichier)
14     for ligne in reader:
15         print(ligne)
16
```

Astuce du marchand

Pourquoi CSV c'est génial ?

- On peut l'ouvrir avec Excel, LibreOffice, Google Sheets.
- Tous les outils d'IA (comme Pandas, qu'on verra plus tard) lisent les CSV en une ligne.
- C'est simple, léger, universel.

1.7 Les modes de fichiers à retenir

Mode	Nom	Que fait-il ?
"r"	Read	Lit le fichier. Le fichier doit exister.
"w"	Write	Écrit. Écrase tout contenu existant.
"a"	Append	Ajoute à la fin. Ne supprime rien.
"r+"	Read + Write	Lit et écrit.

1.8 Application pratique : Le carnet de ventes de Youssouf

On va créer un programme complet qui :

- Demande à Youssouf les ventes du jour.
- Les ajoute dans un fichier `ventes.csv`.
- Relit le fichier et affiche le total des beignets vendus.

```

1  import csv
2
3  def ajouter_vente(jour, nature, chocolat, confiture):
4      # Ajoute une ligne au fichier CSV
5      with open("ventes.csv", "a", newline="") as fichier:
6          writer = csv.writer(fichier)
7          writer.writerow([jour, nature, chocolat, confiture])
8          print("Vente ajoutée !")
9
10 def lire_ventes():
11     # Lit toutes les ventes
12     with open("ventes.csv", "r") as fichier:
13         reader = csv.reader(fichier)
14         return list(reader)
15
16 # Programme principal
17 jour = input("Quel jour sommes-nous ? ")
18 nature = int(input("Combien de beignets nature ? "))
19 chocolat = int(input("Combien de beignets chocolat ? "))
20 confiture = int(input("Combien de beignets confiture ? "))
21
22 ajouter_vente(jour, nature, chocolat, confiture)
23
24 # Lecture et calcul du total
25 ventes = lire_ventes()
26 total = 0
27 for ligne in ventes[1:]: # On saute l'en-tete
28     total += int(ligne[1]) + int(ligne[2]) + int(ligne[3])
29
30 print("Total de beignets vendus :", total)
31

```

1.9 Exercice : Le journal de Hamadi

À ton tour

Hamadi veut tenir un journal personnel dans un fichier `journal.txt`.

- Créez un programme qui demande à l'utilisateur d'écrire une phrase.
- Ajoutez cette phrase dans le fichier `journal.txt` (sans effacer le reste).
- Relisez le fichier et affichez tout ce qui a été écrit.
- Bonus : ajoutez la date et l'heure avant chaque phrase (utilisez le module `datetime`).

1.10 Conclusion : Python n'oublie plus rien

Grâce aux fichiers, Youssouf peut maintenant garder une trace de toutes ses ventes, même après avoir éteint son ordinateur. Et vous, vous savez :

- ① Ouvrir un fichier en lecture ("r"), écriture ("w") ou ajout ("a").
- ② Utiliser `with` pour ne pas oublier de fermer le fichier.
- ③ Lire et écrire des fichiers CSV, le format roi des données.
- ④ Créer un petit système de sauvegarde pour vos programmes.

2 Les erreurs : quand ça plante (et comment ne pas paniquer)

2.1 Le problème du jour : la division par zéro de Youssef

Youssef a écrit un programme pour calculer le prix de ses beignets. Il demande à l'utilisateur combien de beignets il veut, et il divise le prix total par le nombre de beignets pour avoir le prix unitaire.

Un jour, un client (un peu farceur) tape 0. Et là, catastrophe :

```
1 ZeroDivisionError: division by zero
2
```

Le programme plante. Youssef regarde l'écran, les yeux écarquillés. « Mais pourquoi ça a planté ? J'ai rien fait de mal ! »

Hamadi lui dit : « Youssef, les erreurs, c'est normal. Un programme qui ne plante jamais, ça n'existe pas. La différence entre un débutant et un pro, c'est que le pro **anticipe** les erreurs et les gère. »

2.2 Le duo magique : try et except

En Python, pour gérer les erreurs, on utilise try (essayer) et except (sauf, en cas d'erreur).

```
1 try:
2     # Le code qui pourrait planter
3     resultat = 10 / 0
4 except:
5     # Ce qu'on fait si ça plante
6     print("Oups, une erreur s'est produite !")
7
8 print("Le programme continue...")
9
```

Astuce du marchand

Comment ça marche ?

- Python essaie d'exécuter le code dans le bloc try.
- Si une erreur se produit, il saute directement dans le bloc except.
- Si aucune erreur ne se produit, le bloc except est ignoré.
- Dans tous les cas, le programme **ne plante pas** et continue après.

2.3 Attraper des erreurs spécifiques

Dans l'exemple précédent, on attrape **toutes** les erreurs. Mais parfois, on veut savoir **quel type** d'erreur s'est produite, pour réagir différemment.

Python a des noms pour chaque type d'erreur :

- ZeroDivisionError : division par zéro.
- ValueError : mauvaise valeur (par exemple, transformer "abc" en nombre).
- FileNotFoundError : fichier introuvable.
- TypeError : mauvais type (par exemple, additionner un texte et un nombre).
- IndexError : index hors limites (par exemple, demander le 10ème élément d'une liste de 3).
- KeyError : clé introuvable dans un dictionnaire.

```

1  try:
2  nombre = int(input("Donne un nombre : "))
3  resultat = 100 / nombre
4  print("Resultat :", resultat)
5  except ZeroDivisionError:
6  print("Erreur : on ne peut pas diviser par zero !")
7  except ValueError:
8  print("Erreur : ce n'est pas un nombre valide !")
9

```

Astuce du marchand

L'ordre des `except` est important. Si vous mettez un `except` général (sans nom) avant un `except` spécifique, le général attrapera tout et le spécifique ne sera jamais atteint. Mettez toujours les erreurs les plus précises en premier.

2.4 Le bloc `else` : si tout va bien

On peut ajouter un bloc `else` qui s'exécute **seulement** si aucune erreur ne s'est produite.

```

1  try:
2  nombre = int(input("Donne un nombre : "))
3  resultat = 100 / nombre
4  except ZeroDivisionError:
5  print("Division par zero, impossible !")
6  except ValueError:
7  print("Ce n'est pas un nombre !")
8  else:
9  print("Tout s'est bien passe. Resultat :", resultat)
10

```

2.5 Le bloc `finally` : quoi qu'il arrive

Le bloc `finally` s'exécute **toujours**, qu'il y ait eu une erreur ou non. C'est utile pour fermer un fichier, par exemple.

```

1  fichier = open("ventes.txt", "r")
2  try:
3  contenu = fichier.read()
4  print(contenu)
5  except FileNotFoundError:
6  print("Le fichier n'existe pas !")
7  finally:
8  fichier.close()
9  print("Fichier ferme, quoi qu'il arrive.")
10

```

Astuce du marchand

try / except / else / finally : la totale

- try : on essaie.
- except : on attrape l'erreur.
- else : si pas d'erreur.
- finally : dans tous les cas.

Vous n'êtes pas obligé de tout utiliser à chaque fois. Utilisez ce qui est nécessaire.

2.6 Lever ses propres erreurs : raise

Parfois, vous voulez provoquer une erreur vous-même, parce que quelque chose ne respecte pas vos règles. Par exemple, si Youssouf reçoit une commande négative, il veut lever une erreur.

```
1  def commander_beignets(quantite):
2      if quantite < 0:
3          raise ValueError("On ne peut pas commander un nombre negatif de beignets
4      !")
5      if quantite > 100:
6          raise ValueError("Trop de beignets, Youssouf ne peut pas suivre !")
7          print(f"Commande de {quantite} beignets validee.")
8
9      # Test
10     try:
11         commander_beignets(-5)
12     except ValueError as e:
13         print("Erreur attrapee :", e)
```

Astuce du marchand

raise = "je leve une erreur". Quand vous utilisez raise, vous décidez qu'une situation est anormale et vous arrêtez le programme (ou vous laissez un except l'attraper). C'est une façon de dire : « Stop, ça ne va pas ! »

2.7 Application pratique : la calculatrice increvable

On va créer une calculatrice qui ne plante **jamais**, quoi que l'utilisateur tape.

```
1  def calculatrice():
2      print("Bienvenue dans la calculatrice increvable !")
3      try:
4          nombre1 = float(input("Premier nombre : "))
5          nombre2 = float(input("Deuxieme nombre : "))
6          operation = input("Operation (+, -, *, /) : ")
7
8          if operation == "+":
9              resultat = nombre1 + nombre2
10         elif operation == "-":
11             resultat = nombre1 - nombre2
12         elif operation == "*":
13             resultat = nombre1 * nombre2
14         elif operation == "/":
15             resultat = nombre1 / nombre2
```

```

16         else:
17             print("Operation inconnue !")
18
19
20     print("Resultat :", resultat)
21
22     except ValueError:
23         print("Erreur : il faut entrer des nombres valides.")
24     except ZeroDivisionError:
25         print("Erreur : division par zero impossible.")
26     except Exception as e:
27         print("Erreur inattendue :", e)
28     finally:
29         print("Merci d'avoir utilise la calculette !")
30
31 calculette()
32

```

2.8 Exercice : Le formulaire de commande blindé

À ton tour

Reprenez le formulaire de commande de beignets (ou créez-en un nouveau). Écrivez un programme qui :

- Demande le nombre de beignets.
- Demande la saveur (texte).
- Vérifie que le nombre est bien un entier positif. Sinon, lève une erreur `ValueError` avec un message clair.
- Vérifie que la saveur est dans la liste `["nature", "chocolat", "confiture"]`. Sinon, lève une erreur.
- Attrape toutes ces erreurs avec `try/except` et affiche un message sympa à l'utilisateur.
- Utilise `finally` pour dire « Merci, à bientôt ! » à la fin.

2.9 Conclusion : vous êtes un dompteur d'erreurs

Maintenant, vous savez :

- ① Utiliser `try` et `except` pour ne pas planter.
- ② Attraper des erreurs spécifiques (`ZeroDivisionError`, `ValueError`...).
- ③ Utiliser `else` (si tout va bien) et `finally` (dans tous les cas).
- ④ Lever vos propres erreurs avec `raise`.

Un programme qui gère ses erreurs, c'est comme un pêcheur qui prévoit la tempête : il ne coule pas. Vous êtes maintenant un programmeur prudent et solide.

3 Les objets et les classes : le monde des grands

3.1 Le problème du jour : Youssouf veut embaucher

Youssouf a tellement de succès qu'il ne peut plus gérer son stand tout seul. Il veut embaucher des vendeurs. Chaque vendeur a :

- Un **nom**.
- Un **âge**.
- Un **salaire**.
- Une **specialite** (nature, chocolat, confiture).

Et chaque vendeur sait faire des choses :

- **Se présenter** (« Bonjour, je m'appelle X »).
- **Vendre un beignet** (et encaisser l'argent).
- **Prendre une pause** (et manger un beignet gratuit).

Youssef pourrait créer un dictionnaire pour chaque vendeur, mais au bout de 10 vendeurs, ça devient le bazar. Hamadi lui dit : « Il te faut une **classe**. C'est comme un moule à gâteaux : tu définis la forme une fois, et tu peux créer autant de gâteaux que tu veux. »

3.2 Une classe, c'est un moule à gâteaux

Une **classe**, c'est un plan, un modèle, un moule. Elle décrit ce qu'un objet **aura** (ses attributs) et ce qu'il **saura faire** (ses méthodes).

Un **objet**, c'est une instance de la classe. C'est le gâteau fabriqué à partir du moule.

```

1  # Definition de la classe Vendeur
2  class Vendeur:
3      def __init__(self, nom, age, salaire, specialite):
4          self.nom = nom
5          self.age = age
6          self.salaire = salaire
7          self.specialite = specialite
8
9      def se_presenter(self):
10         print(f"Bonjour, je m'appelle {self.nom}, j'ai {self.age} ans.")
11
12     def vendre(self, quantite):
13         gain = quantite * 400
14         print(f"{self.nom} a vendu {quantite} beignets et a encaisse {gain} F.")
15         return gain
16

```

Astuce du marchand

Les mots-clés à retenir :

- **class** : pour définir une classe. Le nom commence par une majuscule (convention).
- **__init__** : le **constructeur**. C'est la fonction qui est appelée automatiquement quand on crée un objet. Elle initialise les attributs.
- **self** : c'est l'objet lui-même. Quand on écrit `self.nom`, on dit « le nom de cet objet précis ». C'est comme dire « ma main » au lieu de « la main ».
- **def** : pour définir une méthode (une fonction qui appartient à la classe).

3.3 Créer des objets : la fabrication des gâteaux

Une fois la classe définie, on peut créer autant d'objets qu'on veut. C'est comme utiliser le moule pour faire des gâteaux.

```

1  # Creation de deux vendeurs (deux objets)
2  youssouf = Vendeur("Youssouf", 17, 50000, "chocolat")
3  hamadi = Vendeur("Hamadi", 18, 60000, "nature")
4
5  # Utilisation des methodes
6  youssouf.sePresenter()
7  hamadi.sePresenter()
8
9  youssouf.vendre(5)
10 hamadi.vendre(3)
11

```

Résultat :

```

1  Bonjour, je m'appelle Youssouf, j'ai 17 ans.
2  Bonjour, je m'appelle Hamadi, j'ai 18 ans.
3  Youssouf a vendu 5 beignets et a encaisse 2000 F.
4  Hamadi a vendu 3 beignets et a encaisse 1200 F.
5

```

Astuce du marchand

Classe vs Objet :

- Vendeur : c'est la classe, le moule, le concept.
- youssouf et hamadi : ce sont des objets, des instances, des gâteaux.
- On peut créer 1000 objets à partir d'une seule classe. C'est ça la puissance.

3.4 Les attributs : ce que l'objet a

Les attributs sont les caractéristiques de l'objet. On les définit dans le constructeur `__init__` avec `self.nom_attribut`.

```

1  class Beignet:
2      def __init__(self, saveur, prix, stock):
3          self.saveur = saveur
4          self.prix = prix
5          self.stock = stock
6
7  # Creation d'un beignet
8  b1 = Beignet("chocolat", 500, 20)
9  print(b1.saveur)    # chocolat
10 print(b1.prix)      # 500
11 print(b1.stock)    # 20
12

```

3.5 Les méthodes : ce que l'objet sait faire

Les méthodes sont les fonctions de la classe. Elles prennent toujours `self` en premier paramètre (pour savoir sur quel objet elles agissent).

```

1  class Beignet:
2      def __init__(self, saveur, prix, stock):
3          self.saveur = saveur
4          self.prix = prix
5          self.stock = stock
6
7      def vendre(self, quantite):

```

```

8         if quantite > self.stock:
9             print("Stock insuffisant !")
10            return 0
11            self.stock -= quantite
12            gain = quantite * self.prix
13            print(f"Vendu {quantite} beignets {self.saveur}. Gain : {gain} F")
14            return gain
15
16        def reapprovisionner(self, quantite):
17            self.stock += quantite
18            print(f"Stock de {self.saveur} : {self.stock}")
19
20        # Test
21        b = Beignet("chocolat", 500, 10)
22        b.vendre(3)           # Vendu 3, stock restant 7
23        b.vendre(15)         # Stock insuffisant !
24        b.reapprovisionner(20) # Stock : 27
25

```

3.6 L'héritage : quand un objet est un cas particulier d'un autre

Imaginez que Youssef veuille créer un **Vendeur en chef**. C'est un vendeur, mais avec des pouvoirs supplémentaires : il peut embaucher d'autres vendeurs.

En Python, on dit que VendeurEnChef **hérite** de Vendeur. Il a tout ce que le vendeur a, plus ses propres trucs.

```

1    class Vendeur:
2        def __init__(self, nom, age):
3            self.nom = nom
4            self.age = age
5
6        def sePresenter(self):
7            print(f"Je suis {self.nom}, {self.age} ans.")
8
9    class VendeurEnChef(Vendeur):
10        def __init__(self, nom, age, equipe):
11            super().__init__(nom, age) # Appelle le constructeur du parent
12            self.equipe = equipe
13
14        def embaucher(self, vendeur):
15            self.equipe.append(vendeur)
16            print(f"{self.nom} a embauche {vendeur.nom}.")
17
18    # Test
19    youssouf = Vendeur("Youssef", 17)
20    hamadi = VendeurEnChef("Hamadi", 25, [])
21
22    hamadi.sePresenter() # Methode heritee
23    hamadi.embaucher(youssouf) # Methode propre
24

```

Astuce du marchand

L'héritage, c'est comme une famille.

- L'enfant hérite des yeux de son père (attributs) et de sa façon de marcher (méthodes).
- Mais l'enfant a aussi ses propres talents.
- `super()` permet d'appeler le constructeur du parent pour ne pas répéter le code.

3.7 Pourquoi c'est indispensable pour l'IA ?

Parce que tous les modèles d'IA sont des **classes** :

- Un **réseau de neurones** est une classe avec des attributs (poids, biais) et des méthodes (entraîner, prédire).
- Une **image** est un objet avec des attributs (largeur, hauteur, pixels) et des méthodes (redimensionner, afficher).
- Un **chatbot** est un objet avec des attributs (nom, personnalité) et des méthodes (répondre, apprendre).

Quand vous verrez du code comme `model = ResNet50()`, vous saurez que `ResNet50` est une classe, et `model` est un objet.

3.8 Application pratique : le stand de Youssef en POO

On va recréer le stand de Youssef avec des classes.

```
1 class Beignet:
2     def __init__(self, saveur, prix, stock):
3         self.saveur = saveur
4         self.prix = prix
5         self.stock = stock
6
7     def vendre(self, quantite):
8         if quantite > self.stock:
9             print(f"Plus assez de {self.saveur} !")
10            return 0
11            self.stock -= quantite
12            return quantite * self.prix
13
14 class Stand:
15     def __init__(self, nom, proprietaire):
16         self.nom = nom
17         self.proprietaire = proprietaire
18         self.beignets = []
19         self.caisse = 0
20
21     def ajouter_beignet(self, beignet):
22         self.beignets.append(beignet)
23
24     def vendre(self, saveur, quantite):
25         for b in self.beignets:
26             if b.saveur == saveur:
27                 gain = b.vendre(quantite)
28                 self.caisse += gain
29                 return gain
30         print(f"Saveur {saveur} inconnue.")
31         return 0
```

```

32
33     def bilan(self):
34         print(f"Stand {self.nom} - Caisse : {self.caisse} F")
35         for b in self.beignets:
36             print(f"    {b.saveur} : {b.stock} en stock")
37
38     # Test
39     stand = Stand("Chez Youssouf", "Youssouf")
40     stand.ajouter_beignet(Beignet("nature", 300, 20))
41     stand.ajouter_beignet(Beignet("chocolat", 500, 15))
42     stand.ajouter_beignet(Beignet("confiture", 450, 10))
43
44     stand.vendre("chocolat", 3)
45     stand.vendre("nature", 5)
46     stand.vendre("fromage", 2)    # Inconnu
47
48     stand.bilan()
49

```

3.9 Exercice : Le lycée de Mutsamoudou en POO

À ton tour

Créez une classe `Eleve` avec :

- Des attributs : `nom`, `age`, `classe`, `notes` (une liste).
- Une méthode `sePresenter()` qui affiche « Je suis X, en classe Y ».
- Une méthode `ajouter_note(note)` qui ajoute une note à la liste.
- Une méthode `moyenne()` qui calcule et retourne la moyenne des notes.

Créez ensuite 3 élèves (Youssouf, Hamadi, Hadidja), ajoutez-leur des notes, et affichez leur moyenne. **Bonus** : Créez une classe `Classe` qui contient une liste d'élèves et une méthode `moyenne_classe()` qui calcule la moyenne de tous les élèves.

3.10 Conclusion : vous êtes un architecte

Avec les classes et les objets, vous ne pensez plus en termes de fonctions éparpillées, mais en termes de **mondes** et d'**objets** qui interagissent. C'est la façon dont les vrais programmeurs construisent des logiciels, et c'est **exactement** comme ça que l'IA est structurée.

Vous savez maintenant :

- ① Définir une classe avec `class`.
- ② Créer des objets à partir d'une classe.
- ③ Utiliser `__init__` pour initialiser les attributs.
- ④ Définir des méthodes qui utilisent `self`.
- ⑤ Faire de l'héritage avec `super()`.